

ASSIGNMENT 2

IMPLEMENT THE FOLLOWING CLASS & INTERFACE INHERITANCE RELATIONSHIPS

Date: 17/09/2026 Marks: 5

1. ASSIGNMENT TITLE

Implementation of Class and Interface Inheritance Relationships in Java

2. OBJECTIVE

To understand and implement different inheritance relationships between classes and interfaces in Java using the **extends** and **implements** keywords, and to identify the relationships supported and not supported by Java.

3. INTRODUCTION

Inheritance is an important Object-Oriented Programming concept that allows a class or interface to acquire members from another class or interface. In Java, the **extends** keyword is used when a class extends a class or an interface extends another interface. The **implements** keyword is used when a class implements one or more interfaces.

Java does not allow a class to extend more than one class. However, Java supports multiple inheritance of type through interfaces. The eight relationships given in the assignment are therefore checked against Java's inheritance rules, and invalid relationships are explicitly documented.

4. EIGHT RELATIONSHIPS + EXPLANATION

1. C1 extends C2 — VALID

A class C1 can extend one class C2. This is ordinary single inheritance.

2. C1 extends C2, C3 — INVALID

A Java class cannot extend two classes. Multiple class inheritance is not supported.

3. C1 implements I1 — VALID

A class C1 can implement an interface I1 and provide implementations for its methods.

4. C1 implements I1, I2 — VALID

A class can implement multiple interfaces, which provides multiple inheritance through interfaces.

5. C1 implements C2 extends I1 — INVALID AS WRITTEN / CORRECT FORM SHOWN

The written order is not valid Java. If the intended relationship is a class extending C2 and implementing I1, the valid form is: class C1 extends C2 implements I1.

6. I1 extends I2 — VALID

An interface can extend another interface and inherit its abstract/default members.

7. I1 implements C1 — INVALID

An interface cannot implement a class. A class implements an interface; an interface extends another interface.

8. I1 extends I2, I3 — VALID

An interface can extend multiple interfaces.

5. COMPLETE CODE WITH COMMENTS

The following consolidated source code demonstrates all eight assignment relationships. Valid relationships are implemented normally. The two invalid relationships are represented through explanatory comments and their valid Java alternatives, so the complete program remains compilable.

```
/*
 * Assignment 2: Class and Interface Inheritance Relationships
 *
 * Relationships covered:
 * 1. C1 extends C2
 * 2. C1 extends C2, C3          -> INVALID in Java
 * 3. C1 implements I1
 * 4. C1 implements I1, I2
 * 5. C1 implements C2 extends I1 -> INVALID as written;
 *                               correct form: C1 extends C2 implements I1
 * 6. I1 extends I2
 * 7. I1 implements C1          -> INVALID in Java
 * 8. I1 extends I2, I3
 */

public class Assignment2InheritanceDemo {

    // =====
    // CASE 1: C1 extends C2
    // =====
    // C1 is a child class and C2 is its parent class.
    static class C2_Case1 {
        void displayC2() {
            System.out.println("Method of C2");
        }
    }

    static class C1_Case1 extends C2_Case1 {
        void displayC1() {
            System.out.println("Method of C1");
        }
    }

    // =====
    // CASE 2: C1 extends C2, C3
    // =====
    // INVALID: Java does not allow a class to extend two classes.
    //
    // The following statement would cause a compilation error:
    // class C1 extends C2, C3 { }
    //
    // Correct alternative: use interfaces for multiple inheritance.

    static class C2_Case2 {
        void methodC2() {
            System.out.println("Method of C2");
        }
    }

    static class C3_Case2 {
        void methodC3() {
            System.out.println("Method of C3");
        }
    }

    interface I2_Case2 {
        void methodI2();
    }

    interface I3_Case2 {
        void methodI3();
    }

    static class C1_Case2 implements I2_Case2, I3_Case2 {
        public void methodI2() {
            System.out.println("Method of I2");
        }

        public void methodI3() {
            System.out.println("Method of I3");
        }
    }

    // =====
    // CASE 3: C1 implements I1
    // =====
    interface I1_Case3 {
        void display();
    }

    static class C1_Case3 implements I1_Case3 {
        public void display() {
            System.out.println("C1 implements I1");
        }
    }

    // =====
    // CASE 4: C1 implements I1, I2
    // =====
    interface I1_Case4 {
        void method1();
    }
}
```

```

}

interface I2_Case4 {
    void method2();
}

static class C1_Case4 implements I1_Case4, I2_Case4 {
    public void method1() {
        System.out.println("Method of I1");
    }

    public void method2() {
        System.out.println("Method of I2");
    }
}

// =====
// CASE 5: C1 implements C2 extends I1
// =====
// INVALID AS WRITTEN.
//
// Correct Java form:
// class C1 extends C2 implements I1
//
// A class may extend one class and implement one or more
// interfaces.

static class C2_Case5 {
    void displayC2() {
        System.out.println("Method of C2");
    }
}

interface I1_Case5 {
    void displayI1();
}

static class C1_Case5 extends C2_Case5 implements I1_Case5 {
    public void displayI1() {
        System.out.println("Method of I1");
    }
}

// =====
// CASE 6: I1 extends I2
// =====
interface I2_Case6 {
    void method2();
}

interface I1_Case6 extends I2_Case6 {
    void method1();
}

static class C1_Case6 implements I1_Case6 {
    public void method1() {
        System.out.println("Method of I1");
    }

    public void method2() {
        System.out.println("Method inherited from I2");
    }
}

// =====
// CASE 7: I1 implements C1
// =====
// INVALID: An interface cannot implement a class.
//
// Wrong form:
// interface I1 implements C1 { }
//
// Valid alternatives:
// class C1 implements I1
// or
// interface I1 extends I2

// =====
// CASE 8: I1 extends I2, I3
// =====
interface I2_Case8 {
    void method2();
}

interface I3_Case8 {
    void method3();
}

interface I1_Case8 extends I2_Case8, I3_Case8 {
    void method1();
}

static class C1_Case8 implements I1_Case8 {
    public void method1() {
        System.out.println("Method of I1");
    }

    public void method2() {
        System.out.println("Method of I2");
    }

    public void method3() {

```

```

        System.out.println("Method of I3");
    }
}

// =====
// MAIN METHOD - TEST CASES
// =====
public static void main(String[] args) {

    System.out.println("=====");
    System.out.println(" CLASS & INTERFACE INHERITANCE");
    System.out.println("=====");

    // TEST CASE 1
    System.out.println("\n--- TEST CASE 1 ---");
    C1_Case1 obj1 = new C1_Case1();
    obj1.displayC2();
    obj1.displayC1();
    System.out.println("C1 extends C2: VALID");

    // TEST CASE 2
    System.out.println("\n--- TEST CASE 2 ---");
    System.out.println("C1 extends C2, C3: INVALID");
    System.out.println("Java does not support multiple class inheritance.");
    System.out.println("Use interfaces to achieve multiple inheritance.");

    // TEST CASE 3
    System.out.println("\n--- TEST CASE 3 ---");
    C1_Case3 obj3 = new C1_Case3();
    obj3.display();
    System.out.println("C1 implements I1: VALID");

    // TEST CASE 4
    System.out.println("\n--- TEST CASE 4 ---");
    C1_Case4 obj4 = new C1_Case4();
    obj4.method1();
    obj4.method2();
    System.out.println("C1 implements I1, I2: VALID");

    // TEST CASE 5
    System.out.println("\n--- TEST CASE 5 ---");
    C1_Case5 obj5 = new C1_Case5();
    obj5.displayC2();
    obj5.displayI1();
    System.out.println("Correct form: C1 extends C2 implements I1");

    // TEST CASE 6
    System.out.println("\n--- TEST CASE 6 ---");
    C1_Case6 obj6 = new C1_Case6();
    obj6.method1();
    obj6.method2();
    System.out.println("I1 extends I2: VALID");

    // TEST CASE 7
    System.out.println("\n--- TEST CASE 7 ---");
    System.out.println("I1 implements C1: INVALID");
    System.out.println("An interface cannot implement a class.");

    // TEST CASE 8
    System.out.println("\n--- TEST CASE 8 ---");
    C1_Case8 obj8 = new C1_Case8();
    obj8.method1();
    obj8.method2();
    obj8.method3();
    System.out.println("I1 extends I2, I3: VALID");

    System.out.println("\n=====");
    System.out.println(" All relationships were tested.");
    System.out.println("=====");
}
}

```

6. TEST CASE 1–8

Test Case	Relationship	Action	Result
1	C1 extends C2	Create C1 object and call inherited/own methods.	VALID
2	C1 extends C2, C3	Check multiple class inheritance.	INVALID
3	C1 implements I1	Create C1 object and call interface method.	VALID
4	C1 implements I1, I2	Create C1 object and call both interface methods.	VALID
5	C1 implements C2 extends I1	Use corrected Java form: C1 extends C2 implements I1.	INVALID as written / VALID corrected form
6	I1 extends I2	Implement I1 and verify methods from I1 and I2.	VALID
7	I1 implements C1	Check interface implementing a class.	INVALID
8	I1 extends I2, I3	Implement I1 and verify methods from all interfaces.	VALID

7. OUTPUT SCREENSHOTS

The following console-style output represents the expected verified execution. For final submission, actual terminal screenshots can be captured after running the program and inserted in the GitHub/PDF documentation.

```
=====
CLASS & INTERFACE INHERITANCE
=====

--- TEST CASE 1 ---
Method of C2
Method of C1
C1 extends C2: VALID

--- TEST CASE 2 ---
C1 extends C2, C3: INVALID
Java does not support multiple class inheritance.
Use interfaces to achieve multiple inheritance.

--- TEST CASE 3 ---
C1 implements I1
C1 implements I1: VALID

--- TEST CASE 4 ---
Method of I1
Method of I2
C1 implements I1, I2: VALID

--- TEST CASE 5 ---
Method of C2
Method of I1
Correct form: C1 extends C2 implements I1

--- TEST CASE 6 ---
Method of I1
Method inherited from I2
I1 extends I2: VALID

--- TEST CASE 7 ---
I1 implements C1: INVALID
An interface cannot implement a class.

--- TEST CASE 8 ---
Method of I1
Method of I2
Method of I3
I1 extends I2, I3: VALID

=====
All relationships were tested.
=====
```

8. VERIFICATION / RESULTS

Case	Verification
1	C1 successfully extends C2 and accesses the parent method.
2	Multiple class inheritance is identified as invalid in Java.
3	C1 successfully implements I1 and provides its method implementation.
4	C1 successfully implements both I1 and I2.
5	The written form is invalid; the valid form C1 extends C2 implements I1 is demonstrated.
6	I1 successfully extends I2, and a class implementing I1 provides both methods.
7	I1 implementing a class is identified as invalid.
8	I1 successfully extends both I2 and I3.

GITHUB FILE STRUCTURE

```
Assignment-2/  
■■■ Case1_C1ExtendsC2.java  
■■■ Case2_C1ExtendsC2C3.java  
■■■ Case3_C1ImplementsI1.java  
■■■ Case4_C1ImplementsI1I2.java  
■■■ Case5_C1ExtendsC2ImplementsI1.java  
■■■ Case6_I1ExtendsI2.java  
■■■ Case7_I1ImplementsC1.java  
■■■ Case8_I1ExtendsI2I3.java  
■■■ Assignment_2_Class_Interface_Inheritance.pdf
```

9. CONCLUSION

Thus, the specified class and interface inheritance relationships were studied and implemented according to Java language rules. The assignment demonstrates the use of **extends** and **implements**, multiple interface inheritance, class-interface combinations, and Java's restriction on multiple class inheritance. The individual Java files can be uploaded to GitHub along with this overall PDF documentation.